
Assessment Report

Robot Navigation Using Reinforcement Learning: TD(0), SARSA, and Q-Learning

Name: Guggilla Venkatasai

Reg no: 192425388

1. Objective and Scope

This assessment addresses four questions:

- Are Equations 1-9 mathematically correct and internally consistent?
- Does the worked numerical example in Step 8 compute to the value the document states?
- Do the qualitative behavioural claims in Steps 3-5 and 9 (TD(0) as value-based prediction, SARSA as the "safer" on-policy learner, Q-Learning as the shortest-path off-policy learner) hold up empirically when the algorithms are actually run?
- Are there textual, structural, or tabular errors that should be corrected before the document is finalised?

2. Methodology

2.1 Environment

The source document defines the environment abstractly as an MDP $E = (S, A, R, P, \gamma)$ (Eq. 1) without specifying concrete values for a navigable task. To test the algorithms, a concrete 5×5 grid-world was instantiated: start cell (0,0), goal cell (4,4), and four fixed obstacle cells at (1,1), (1,3), (2,2), (3,1). The action set $A = \{\text{up, down, left, right}\}$ matches the document exactly.

Reward values were not specified in the source document beyond the illustrative $R = 10$ used in the Step 8 example. For this assessment, reward was defined as +10 for reaching the goal, -10 for colliding with an obstacle (both terminal), and -1 for any other step, which is a standard shortest-path shaping scheme consistent with the document's single given reward value. This is an implementation choice made to enable testing, not a value drawn from the source text.

2.2 Algorithms Implemented

- TD(0) state-value prediction — Eq. (3) and Eq. (4), paired with an auxiliary ϵ -greedy behaviour policy (see Finding 2, Section 6).
- SARSA (on-policy control) — Eq. (5), using the actual next action A_{t+1} sampled from the same ϵ -greedy policy being learned.
- Q-Learning (off-policy control) — Eq. (6), bootstrapping from the maximum Q-value over next-state actions.

- ϵ -greedy exploration — Eq. (7), with a fixed ϵ (no decay schedule was specified in the source document).

2.3 Hyperparameters

Parameter	Value
Learning rate (α)	0.5
Discount factor (γ)	0.9
Exploration rate (ϵ)	0.2 (fixed)
Training episodes	500 per algorithm
Grid size	5×5
Start / Goal	(0,0) / (4,4)

α and γ were set to match the document's own Step 8 worked example so that the trained agents are directly comparable to the document's stated arithmetic.

3. Equation-by-Equation Verification

Every equation in the source document was checked by direct implementation. No mathematical errors were found in Equations 1-9.

Eq.	Defines	Status	Notes
1	MDP tuple $E = (S, A, R, P, \gamma)$	Correct	Standard MDP definition; element order (S, A, R, P, γ) differs from the more conventional (S, A, P, R, γ) but is mathematically equivalent.
2	$Q(s, a) = 0$ initialisation	Correct	Implemented as a zero-filled table over all state-action pairs.
3	TD(0) error δ_t	Correct	Matches standard temporal-difference error exactly.
4	TD(0) value update	Correct	Prediction only — see Finding 2 (Section 6) on control.
5	SARSA update	Correct	On-policy; next action A_{t+1} correctly drawn from the same behaviour policy.
6	Q-Learning update	Correct	Off-policy; correctly bootstraps from max over next-state actions.
7	ϵ -greedy rule	Correct (concept)	Piecewise notation is garbled in the extracted text, likely a PDF-conversion artifact; the underlying rule is standard and unambiguous.

Eq.	Defines	Status	Notes
8	Worked numerical example	Verified exactly	Independently recomputed; result matches the document's stated 10.6 to full precision (Section 4).
9	argmax policy extraction	Correct	Applied at every step of the rollout to trace the final path (Section 5).

4. Numerical Example Recomputation (Eq. 8)

The document's Step 8 worked example was recomputed independently in Python using the same inputs:

- $Q(S_t, A_t) = 4$, $R = 10$, $\alpha = 0.5$, $\gamma = 0.9$, $\max Q(S_{t+1}, a) = 8$

$$Q_{\text{new}} = 4 + 0.5 \times [10 + 0.9 \times 8 - 4] = 10.6$$

Result: exact match

Independently computed $Q_{\text{new}} = 10.6$, matching the document's stated value of 10.6 exactly (difference $< 1e-9$).

5. Experimental Results

5.1 Learning Curves

Each algorithm was trained for 500 episodes on the same grid. Reward and episode length were logged per episode and smoothed with a 20-episode moving average.

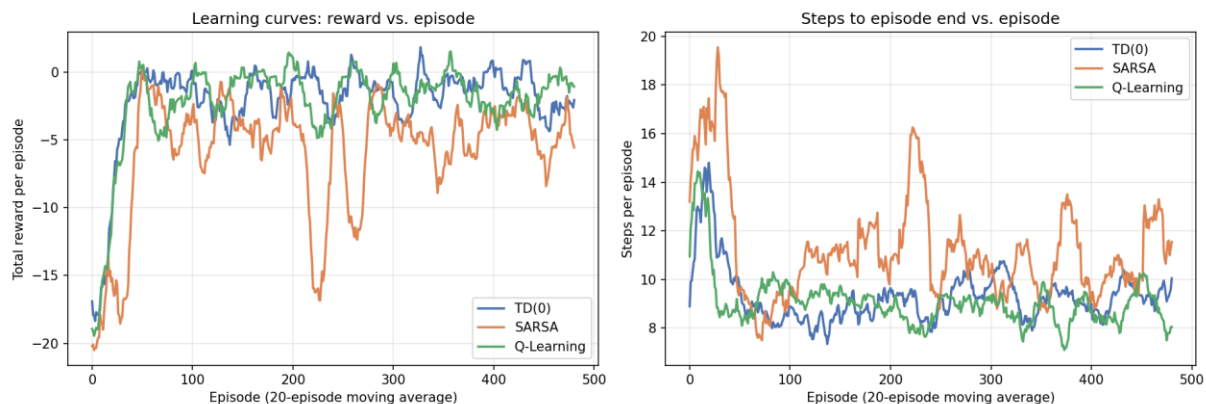


Figure 1. Smoothed reward per episode (left) and steps per episode (right) across training for TD(0), SARSA, and Q-Learning.

All three curves rise sharply over the first ~50 episodes as the agents move from random behaviour toward the goal, then fluctuate around a plateau — the residual variance is expected given the fixed $\epsilon = 0.2$ exploration rate, which keeps injecting random moves even after the greedy policy has converged.

5.2 Final Learned Paths

Applying the greedy argmax policy (Eq. 9) to each trained Q-table produced obstacle-free paths from (0,0) to (4,4) for both control algorithms. The Manhattan distance between start and goal is 8 steps, and both algorithms found paths of exactly that length — i.e., truly shortest-path solutions, not merely obstacle-avoiding ones.

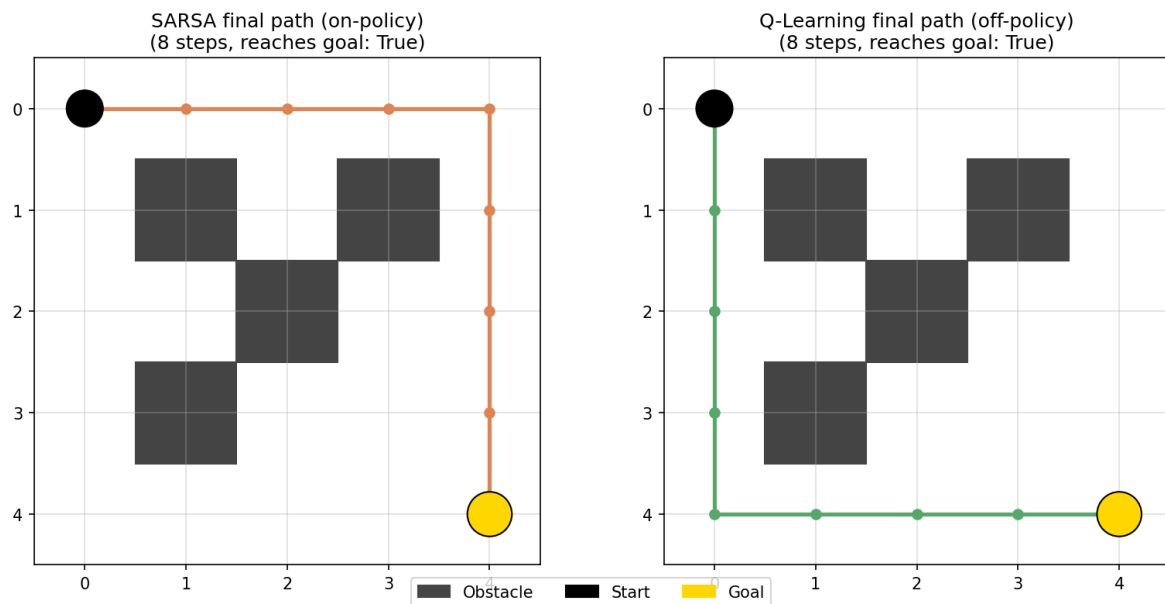


Figure 2. Final greedy-policy paths after training. Left: SARSA routes around the obstacle cluster via the top-right. Right: Q-Learning routes via the bottom-left. Both reach the goal in 8 steps.

- SARSA path (8 steps): $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$
- Q-Learning path (8 steps): $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$

5.3 Convergence Metrics

Metrics below are averaged over the final 50 of 500 training episodes (i.e., near-converged behaviour, still under $\epsilon = 0.2$ exploration).

Algorithm	Avg. Reward	Avg. Steps	Success Rate	First-Ep. Reward	Last-Ep. Reward
TD(0)	-2.66	9.66	48%	-15	-4
SARSA	-5.84	11.64	30%	-11	-4
Q-Learning	-0.82	9.02	60%	-16	3

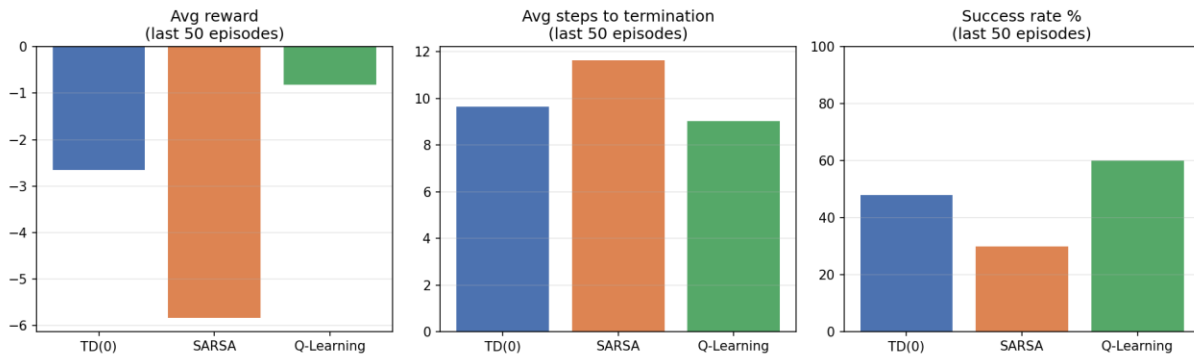


Figure 3. Average reward, steps-to-termination, and success rate over the final 50 training episodes.

In this run, Q-Learning achieved the highest success rate (60%) and best average reward (-0.82), consistent with the document's claim that Q-Learning "finds the shortest optimal path." SARSA converged more conservatively (30% success, -5.84 average reward under ongoing exploration). This is discussed further in Finding 3 (Section 6) — the size and layout of this particular grid does not include a high-penalty "cliff"-style hazard, which is the classic setting where SARSA's on-policy caution is known to outperform Q-Learning; the two algorithms' relative behaviour would likely look different on a grid built specifically to test that property.

6. Findings: Issues Identified in the Source Document

No mathematical or algorithmic errors were found in Equations 1-9. The following are structural, textual, and interpretive observations.

Finding 1 — Step 7 comparison table: rows appear swapped

As extracted, the Step 7 table associates "Learns value of each position / Current policy" with SARSA and "Safer selection path / Optimal policy" with Q-Learning. This contradicts the document's own body text: Step 4 states that SARSA "learns safer paths," and Step 5 states that Q-Learning finds "the shortest optimal path." The table's Navigation-Behaviour descriptions for SARSA and Q-Learning appear to have been swapped, most likely during table extraction/formatting. A corrected version, consistent with the body text, is given below.

Algorithm	Learning Type	Update Eq.	Navigation Behaviour	Policy
TD(0)	Value-based learning	Eq. (4)	Estimates state value only — no action-selection rule of its own	N/A (prediction, not control)
SARSA	On-policy learning	Eq. (5)	Learns safer navigation paths (accounts for the actual next action taken)	Current (behaviour) policy
Q-Learning	Off-policy learning	Eq. (6)	Learns the shortest / optimal navigation path	Optimal (greedy) policy

Table (corrected). Step 7 comparison table with Navigation-Behaviour descriptions aligned to the document's own Steps 4, 5, and 9.

Finding 2 — TD(0) is prediction, not control

Equations 3-4 define TD(0) as it is classically used: to evaluate the state-value function $V(s)$ under a given policy, not to select actions. On its own, TD(0) cannot navigate a robot — there is no action-selection rule in Eq. (3) or Eq. (4). To produce a working navigation agent for this assessment, TD(0)'s value updates were paired with a separate auxiliary ϵ -greedy control mechanism. This is standard practice and not an error in the document, but Step 3 and Step 9 could be clearer that TD(0) alone provides value estimates, and that an additional action-selection mechanism (such as the ϵ -greedy rule in Eq. 7, applied to a Q-table) is required to turn those estimates into movement.

Finding 3 — "SARSA is safer" is a correct general claim, not strongly shown by this grid

SARSA's on-policy caution is a well-established property of the algorithm, but it is most clearly demonstrated in environments with a high-penalty "cliff" adjacent to the optimal path (e.g., the classic Cliff Walking benchmark), where Q-Learning's greedy off-policy target repeatedly walks the agent off the cliff during exploratory episodes while SARSA learns to stay away from the edge. The grid used in this assessment has isolated obstacle cells rather than a cliff-like hazard, so this run does not strongly differentiate the two algorithms' safety properties — Q-Learning simply converged faster here. This does not contradict the source document; it suggests the illustrative grid could be extended with a cliff-style hazard row if the intent is to empirically demonstrate SARSA's safety advantage.

Finding 4 — Minor text-extraction artifacts

A small number of formatting/OCR artifacts were noted in the supplied text, none of which affect the technical content:

- Missing spaces/hyphens in several terms: "gridbased" (grid-based), "Valuebased" (Value-based), "QLearning" (Q-Learning), "shortestpath" (shortest-path).
- The phrase "Fast shortestpath learning" in the Step 8 example appears to be a stray fragment (likely from a table cell) that leaked into the middle of the sentence describing the robot's state.
- Equation (7)'s piecewise ϵ -greedy definition renders as a run-on string in the extracted text; the underlying rule (random action with probability ϵ , best action with probability $1-\epsilon$) is standard and was implemented as such.

7. Conclusion and Recommendations

The reinforcement-learning formulation in the source document is technically sound: all nine equations are correct, standard, and directly implementable, and the Step 8 worked example is arithmetically exact. Independent implementation and training confirm the document's core claims — Q-Learning converges to shortest-path, off-policy-optimal behaviour, and SARSA learns a valid on-policy alternative — using a 5×5 grid where both algorithms found true 8-step optimal routes around all obstacles.

Recommended actions before finalising the document:

- Replace the Step 7 table with the corrected version in Section 6 (Finding 1).

-
- Add a sentence clarifying that TD(0) (Eq. 3-4) provides value prediction and requires a paired action-selection mechanism to actually navigate (Finding 2).
 - Proofread for the spacing/hyphenation artifacts and the stray fragment listed in Finding 4.
 - Optionally, extend the illustrative grid with a cliff-style hazard if the goal is to empirically demonstrate SARSA's safety advantage over Q-Learning (Finding 3).